Project Documentation: RAG-Based Customer Support Assistant

1. High-Level Design (HLD)

System Overview

This project is a Retrieval-Augmented Generation (RAG) system designed for Customer Support. It allows users to query information from a specific PDF knowledge base. The system uses a graph-based workflow to ensure structured execution and decision-making.

Architecture Components

- **User Interface:** Command Line Interface (CLI) for user queries.
- **Document Ingestion:** `PyPDFLoader` for extracting text from PDF files.
- **Text Splitter:** `RecursiveCharacterTextSplitter` to create manageable chunks.
- **Embedding Model: Google Generative AI Embeddings** (`models/embedding-001`).
- **Vector Database: ChromaDB** for high-performance similarity search.
- **Orchestration: LangGraph** to manage state and control flow.
- **LLM Engine: Gemini 1.5 Flash** for contextual answer generation.

Technology Stack

Component	Technology
Programming Language	Python 3.10+
GenAI API	Google Gemini API
Framework	LangChain & LangGraph
Vector Store	ChromaDB

---

2. Low-Level Design (LLD)

Module Descriptions

1. **Ingestion Module:** * Reads `data/manual.pdf`.
    - Chunk Size: 700 characters | Overlap: 100 characters.
2. **Retrieval Module:**
    - Converts user query into embeddings.
    - Performs similarity search in ChromaDB.
    - Returns top 3 relevant chunks.
3. **Graph Logic (State Management):**
    - **Node 'Retriever':** Fetches context.
    - **Node 'Generator':** Processes context + query to produce an answer.
4. **HITL (Human-in-the-Loop) Design:**
    - **Condition:** If `context` is null or empty.
    - **Escalation:** The system returns a fallback message: *"I cannot find this info, escalating to a human agent."*

Data Schema

Python
```
class AgentState(TypedDict):
    question: str
    answer: str
    context: List[str]
    needs_human: bool
```

---

3. Technical Documentation & Design Decisions

Design Decisions

- **Why Gemini 1.5 Flash?** It provides a perfect balance between speed and reasoning capabilities, making it ideal for real-time support.
- **Why LangGraph?** Unlike simple chains, LangGraph allows for stateful, multi-step transitions and easy integration of HITL logic.
- **Chunking Strategy:** 700 characters ensure that enough context is passed to the LLM without exceeding token limits or losing semantic meaning.

Challenges & Trade-offs

- **Accuracy vs. Speed:** We chose Gemini 1.5 Flash to ensure the assistant responds in under 2 seconds while maintaining high retrieval accuracy.
- **No Context Found:** Instead of letting the LLM "hallucinate" (make up answers), we implemented a strict check to trigger human escalation.

Testing Strategy

- **Sample Query:** "What is the refund policy?"
- **Expected Result:** The system should pull the specific refund paragraph from the PDF.
- **Edge Case:** Querying something not in the PDF should trigger the "Escalating to human" message.